



Guardrails para Vibe Coding no Git

Como evitar que a IA cometa erros críticos de Git & GitHub

Cenário 1: Cursor

Cenário 2: VSCode + Claude Code

Leonardo Camilo • Technical Training

O Problema do Vibe Coder no Git

A IA executa o que você pede — mas não sabe o que você NÃO pediu

O que o usuário imagina que acontece

"Salva o código no GitHub"

→ add + commit + push ✓

"Atualiza o projeto"

→ pull antes de editar ✓

"Não manda a senha"

→ usa .gitignore ✓



O que a IA faz sem guardrails

"Salva o código"

→ só faz commit, não push ✗

"Atualiza o projeto"

→ edita sem dar pull antes ✗

"Cria arquivo .env"

→ commita a senha junto ✗

A Solução: 3 Camadas de Guardrails

Cada camada resolve uma categoria diferente de erro

1



Arquivos de Regras Globais

`Cursor User Rules / ~/.claude/CLAUDE.md`

Instrui a IA sobre COMO operar no Git em QUALQUER projeto. Configurado uma única vez — automático para sempre.

2



Proteção de Segredos

`.gitignore_global + .env pattern`

Configurado globalmente via git config. Impede que senhas e tokens cheguem ao GitHub em qualquer projeto do computador.

3



Git Hooks

`pre-commit + commit-msg`

Scripts executados automaticamente antes de cada commit. Bloqueiam erros antes que aconteçam — independente da IA.

Cenário 1

Cursor

AI-first code editor

No Cursor, o mecanismo principal de controle é o arquivo

Cursor Settings → Rules

User Rules (global – todos os projetos)

Configurado UMA VEZ no Cursor Settings → Rules → User Rules. Vale para TODOS os projetos automaticamente — sem precisar criar nenhum arquivo.



Ativo globalmente — funciona em qualquer projeto que você abrir



Lista arquivos que NUNCA podem ser commitados



Especifica mensagem de commit obrigatória

.cursorrules — Regras de Git

Cenário 1 • Cursor



```
# .cursorrules — Git Workflow Rules
```

GIT OPERATIONS — MANDATORY

When the user asks to 'save', 'push',
'send to github' or 'update repo',
ALWAYS execute ALL of these steps:

1. `git add -A`
2. `git commit -m "<type>: <description>"`
3. `git push origin <current-branch>`

NEVER stop at commit without pushing.
ALWAYS confirm push success to user.



Commit sem Push = Invisível

Código commitado mas não enviado fica só no PC local. Outros não veem, backup não existe. A IA DEVE sempre terminar com push.



Ambiguidade Fatal

"Salva meu código" pode virar apenas git add. Defina explicitamente que toda operação de 'salvar' = add + commit + push.



Confirmação Obrigatória

A IA deve reportar: "Push concluído — commit abc123 em origin/main". O usuário precisa saber que funcionou.

.cursorrules — Segurança e Mensagens

Cenário 1 • Cursor



SECURITY — NEVER COMMIT THESE

```
FORBIDDEN files — NEVER add/commit:  
.env .env.* *.key *.pem *.p12  
config/secrets.* credentials.*  
*password* *token* *secret*
```

If user asks to commit any of these,
REFUSE and explain the risk.

COMMIT MESSAGE FORMAT

ALWAYS use Conventional Commits:

```
feat: add ENB parser for XML  
fix: correct UTF-8 encoding  
docs: update README examples  
refactor: split export logic
```



Senhas no Git = Desastre

Uma vez commitada, a senha fica no histórico PARA SEMPRE — mesmo se deletar o arquivo depois. Tokens vazados no GitHub são detectados por bots em segundos.



Conventional Commits

Padrão que facilita leitura do histórico e geração automática de changelog. A IA deve SEMPRE usar: feat / fix / docs / refactor / chore / test.

.cursorrules — Pull Antes de Trabalhar

Cenário 1 • Cursor



WORKFLOW — START OF SESSION

When starting any new task or the user opens the project, ALWAYS run:

```
git status
git pull origin <current-branch>
```

BEFORE making any code changes.

STATUS CHECK

After every git operation, run:

```
git status
git log --oneline -3
```

Report results to user.

✗ Sem Guardrail

User: 'adiciona feature X'

IA: (edita direto sem pull)

IA: git add . && git commit -m '...'

IA: git push → CONFLICT ✗

Histórico divergente, merge manual

✓ Com Guardrail

User: 'adiciona feature X'

IA: git pull origin main

IA: (edita arquivos)

IA: add + commit + push ✓

Push limpo, sem conflito

Cenário 2

VSCode + Claude Code

No Claude Code, o mecanismo principal é o arquivo global

`~/.claude/CLAUDE.md`

(arquivo global – todos os projetos)

Diferença chave em relação ao Cursor: o CLAUDE.md global fica em ~/.claude/ e é lido automaticamente em TODOS os projetos. Também suporta CLAUDE.md local por projeto para regras específicas.



CLAUDE.md — instruções persistentes de comportamento



CLI flags — --allowedTools, --dangerouslySkipPermissions



Hooks MCP — permite acionar scripts de validação

CLAUDE.md — Regras de Git

Cenário 2 • Claude Code



```
# CLAUDE.md — Project Rules
```

GIT WORKFLOW

MANDATORY sequence for any save/update:

```
git pull origin $(git branch --show-current)
# ... make changes ...
git add -A
git status # verify before commit
git commit -m "type: description"
git push origin $(git branch --show-current)
git log --oneline -3 # confirm
```

NEVER use `git push --force`

ALWAYS show user final git status

Vantagens exclusivas do Claude Code



CLI com `--allowedTools`

Restringe quais ferramentas a IA pode usar. Ex: bloquear git push sem confirmação explícita do usuário.



Modo Interativo por Padrão

Claude Code pede confirmação antes de executar comandos destrutivos — comportamento mais seguro que o Cursor.



Audit Trail Automático

Todo comando executado fica registrado no histórico de sessão — rastreabilidade total do que a IA fez.

CLAUDE.md — Segurança e Secrets

Cenário 2 • Claude Code



SECURITY — CRITICAL RULES

Files FORBIDDEN in any commit:

```
.env .env.* .env.local  
*.key *.pem *.p12 *.pfx  
*secret* *password* *token*  
config/credentials.*
```

If asked to commit any of these:

1. **REFUSE the operation**
2. Explain why it's dangerous
3. Suggest `.gitignore` solution

ALWAYS verify `.gitignore` exists

before first commit in project

.gitignore mínimo obrigatório



```
# Secrets — NEVER commit these  
.env  
.env.*  
*.key  
*.pem  
  
# Python  
__pycache__/  
*.pyc  
venv/ .venv/  
  
# IDE  
.cursor/ .vscode/
```

Git Hooks — A Defesa Infalível

Funciona nos 2 Cenários

Scripts executados AUTOMATICAMENTE pelo Git — antes de qualquer commit.

A IA não tem como contornar — o Git bloqueia se o hook falhar.



```
#!/bin/sh
# .git/hooks/pre-commit

# Block secrets from being committed
if git diff --cached --name-only | \
    grep -qE
    "\.(env|key|pem|p12)$|secret|password|token"; then
    echo "BLOCKED: Sensitive file detected!"
    echo "Remove it and use .gitignore"
    exit 1
fi

# Check commit message format
# (in commit-msg hook)
```



```
#!/bin/sh
# .git/hooks/commit-msg

# Enforce Conventional Commits format
MSG=$(cat "$1")
PATTERN="^(feat|fix|docs|refactor|chore|test)"

if ! echo "$MSG" | grep -qE "$PATTERN"; then
    echo "BLOCKED: Bad commit message!"
    echo "Use: feat|fix|docs|refactor..."
    exit 1
fi
```

```
chmod +x .git/hooks/pre-commit
.git/hooks/commit-msg
```

Cursor vs Claude Code — Comparação

O que cada ferramenta oferece nativamente para guardrails de Git

	Cursor	VSCode + Claude Code
Configuração global	✓ Settings → User Rules	✓ ~/.claude/CLAUDE.md
Arquivo de regras	✓ User Rules (global)	✓ CLAUDE.md global + local
Vale para todo projeto	✓ Sim, automático	✓ Sim, automático
Pedir confirmação	✗ Limitado	✓ Nativo e detalhado
Controle de ferramentas	✗ Não disponível	✓ --allowedTools flag
Audit trail	✗ Limitado	✓ Sessão completa
Git Hooks (pre-commit)	✓ Manual por projeto	✓ Manual por projeto
.gitignore global	✓ git config manual	✓ git config manual

Checklist — Setup Completo de Guardrails

Configure UMA ÚNICA VEZ — as regras valem para todos os projetos automaticamente

Cursor	Ambos os Cenários	VSCode + Claude Code
 Cursor Settings → Rules → User Rules	 Criar .gitignore com .env, *.key, *.pem	 Criar ~/.claude/CLAUDE.md (global)
 Colar as regras de Git Workflow (add + commit + push)	 Criar .git/hooks/pre-commit + chmod +x	 Adicionar seção ## GIT WORKFLOW
 Listar arquivos FORBIDDEN para commit	 Criar .git/hooks/commit-msg + chmod +x	 Adicionar seção ## SECURITY com arquivos proibidos
 Definir formato de commit message	 Testar hooks: tentar commit de arquivo .env	 Configurar .gitignore global via git config
 Salvar — ativo em todos os projetos		

Onde Fica Cada Arquivo de Guardrail

Onde ficam os arquivos globais



~/ (pasta home do usuário)

```
├── .cursor/
│   └── settings.json ← User Rules ficam aqui
├── .claude/
│   └── CLAUDE.md ← regras globais do Claude Code
└── .gitignore_global ← bloqueia secrets em tudo
```

meu_projeto/ (qualquer projeto)

```
├── .git/
│   └── hooks/
│       ├── pre-commit ← bloqueia secrets
│       └── commit-msg ← valida mensagem
```

Setup rápido — cole no terminal:



```
# CURSOR — Cole em Settings → Rules → User Rules
# (interface gráfica, não precisa de terminal)
```

```
# CLAUDE CODE — Configure o global:
```

```
mkdir -p ~/.claude
```

```
# Cole as regras em ~/.claude/CLAUDE.md
```

```
# GIT — .gitignore global para todos os projetos:
```

```
git config --global core.excludesfile \
    ~/.gitignore_global
```

```
# Cole o conteúdo em ~/.gitignore_global
```

```
# HOOKS — rodar 1x em cada projeto novo:
```

```
chmod +x .git/hooks/pre-commit
```

```
chmod +x .git/hooks/commit-msg
```

Resumo: Os 5 Guardrails Essenciais

Implemente estes 5 itens e seu vibe coder estará protegido no Git

1



Regras Globais de IA

Cursor: Settings → Rules → User Rules. Claude Code: `~/.claude/CLAUDE.md`. Configurado UMA VEZ, vale para todos os projetos.

2



`.gitignore` global

`git config --global core.excludesfile ~/.gitignore_global`. Bloqueia `.env`, `*.key`, `*.pem` em TODOS os projetos do computador.

3



Git Hook pre-commit

Última barreira antes do commit. Recusa qualquer arquivo com 'secret', 'token' ou 'password' no nome.

4



Git Hook commit-msg

Obriga Conventional Commits: `feat/fix/docs/refactor`. Histórico legível para sempre.

5



Confirmação de Push

A IA DEVE confirmar push com `git log --online -3` e mostrar resultado. Commit sem push = código invisível.